

LoongArch32 双发射 5 级流水线 CPU 设计文档

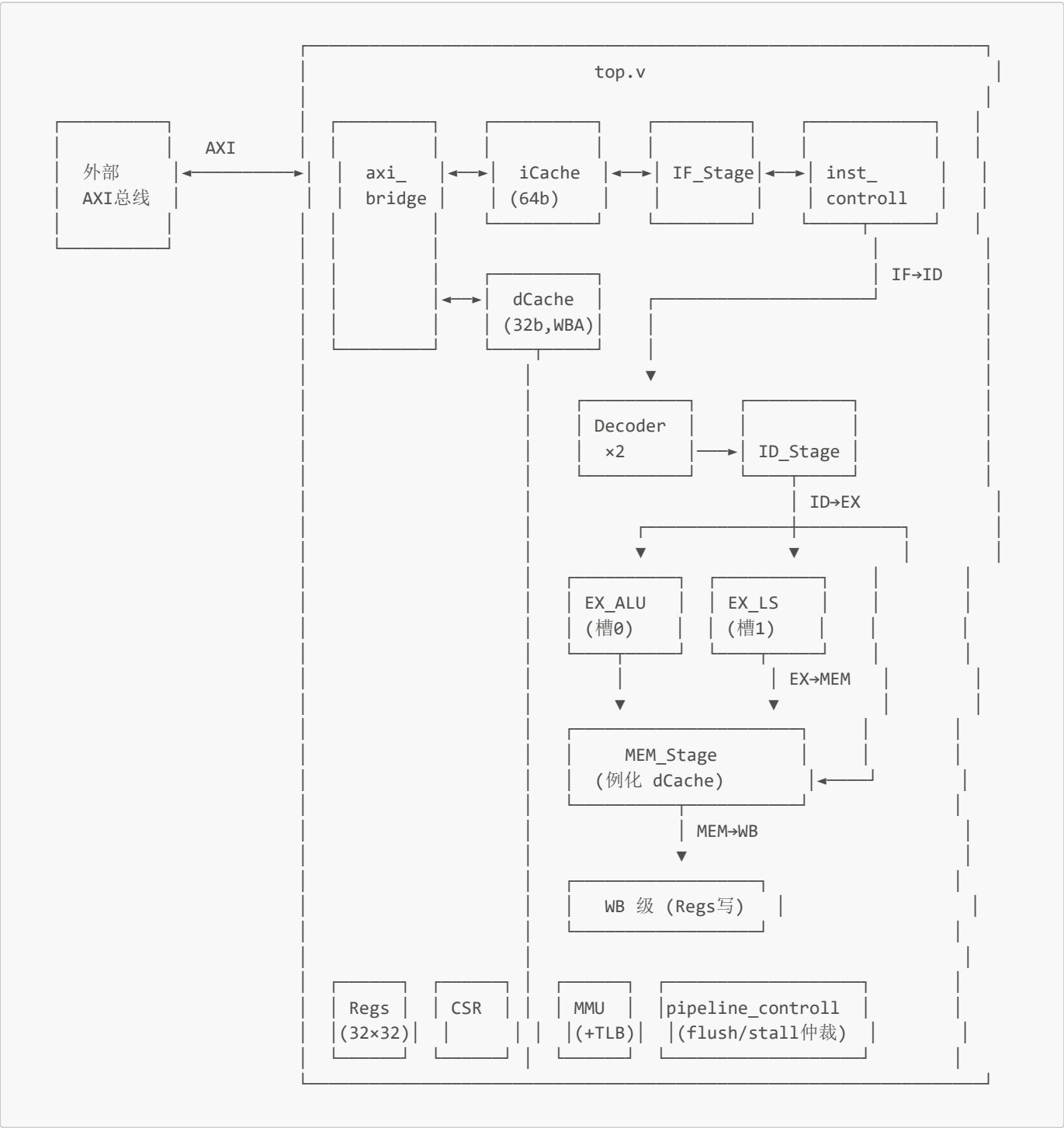
1. 总体架构概览

本设计是一颗 **LoongArch32 (LA32)** 指令集的双发射、五级流水线处理器。流水线阶段为：**IF** → **ID** → **EX** → **MEM** → **WB**。支持两个发射槽位：

- 槽0 (Slot 0)：ALU / Branch / Jump / CSR 指令
- 槽1 (Slot 1)：ALU / Load / Store 指令

工作频率：设计目标为该系列 FPGA 开发板典型频率（~50–100 MHz）。

2. 模块设计接线图



顶层封装：

core_top.v 将 top.v 封装为统一 AXI 接口，提供调试端口（debug0_wb_pc 等）。

3. 按流水线阶段分类的模块功能

3.1 IF 阶段（取指）

模块	文件	主要功能
inst_controll	inst_controll.v	PC 控制核心：管理下一条 PC、处理跳转/异常重定向、处理未对齐取指的指令拼接。当跳转目标 4B 对齐而非 8B 对齐时，将高半指令作为低半发射，并填充 NOP。
IF_Stage	IF_Stage.v	取指阶段封装，内嵌 iCache。输出 64b dual_inst（两条指令对），if_stall 指示 iCache 缺失暂停。
iCache	iCache.v	2 路组相联指令缓存（只读），64B 行大小，数据宽度 64b（双发射）。命中 0 stall，缺失时通过 AXI 逐字填充（每 2 次 32b AXI 读拼装 1 个 64b BRAM 字）。LRU 替换策略。
MMU (IF侧)	mmu.v	将 pc_next 虚地址翻译为物理地址 if_pa，支持直接地址翻译 (DA=1) 和 DMW 映射翻译。

关键数据通路：

```
pc_next → MMU(if_va→if_pa) → IF_Stage/iCache → dual_inst_raw(64b)
          → inst_controll(pc控制 + 指令拼接) → dual_inst(64b) → ID_Stage
```

3.2 ID 阶段（译码 & 发射）

模块	文件	主要功能
Decoder ×2	Decoder.v	LA32 指令译码器。识别 3R 型 ALU、I 型 ALU（含移位立即数）、LU12I.W / PCADDU12I、Load/Store、Branch、Jump (B/BL/JIRL)、CSR 操作、ERTN、SYSCALL/BREAK。输出 opcode、func、立即数、寄存器地址、控制信号（memRead/memWrite/branch/jump/regWrite/alu/load/store/valu）及异常码（INE/BRK/SYS/ADEF）。
ID_Stage	ID_stage.v	发射控制器。核心功能：(1) 冲突检测：双 LS 冲突、双 branch/jump 冲突、RAW 同拍依赖 (dep10) 和上一拍 load→本拍依赖 (dep09/dep19)。(2) 槽位分配：必要时交换高低指令以确保 LS→槽1、Branch/Jump→槽0 (swap_ls / swap_br)。(3) NOP 标记：nop1（低位未发射）、noph（高位未发射）反馈给 inst_controll。(4) 异常仲裁：ADEF（PC 未对齐）> 中断 > 指令异常。
Regs (读口)	Regs.v	32×32b 寄存器文件，r0 硬连线为 0。4 读 2 写端口。ID 阶段组合读出两个槽共 4 个源操作数。

sigBus 控制信号定义：

位	信号	含义
6	valu	EX 结果会被写入寄存器
5	jump	无条件跳转
4	branch	条件分支
3	memRead	Load 指令
2	memWrite	Store 指令

位	信号	含义
1	regWrite	写回寄存器
0	high	双发射时当前槽 PC 是否更高

csrBus 控制信号定义 (17b):

位	信号	含义
16:3	csreg	CSR 寄存器编号
2	xchg	掩码交换写入 CSR
1	csrwr	直接写入 CSR
0	csrrd	读取 CSR

冲突检测与发射规则:

场景	处理方式
双 LS (memRead/memWrite)	冲突, 仅槽1发射 LS, 槽0废弃
双 Branch/Jump/CSR	冲突, 仅槽0发射, 槽1废弃
槽0=LS, 槽1≠LS	交换 (swap_ls), LS→槽1
槽1=Branch/Jump, 槽0≠	交换 (swap_br), →槽0
高位 RAW 依赖低位	高位 NOP, 下拍重发
上一拍 load → 本拍依赖	依赖者 NOP, 下拍重发

3.3 EX 阶段 (执行)

模块	文件	主要功能
EX_ALU	EX_ALU.v	槽0 执行单元。功能: (1) 3R 型 ALU 运算 (ADD/SUB/SLT/SLTU/NOR/AND/OR/XOR/SLL/SRL/SRA/MUL/MULH/MULHU/DIV/MOD/DIVU/MODU); (2) I 型 ALU (ADDI/ANDI/ORI/XORI/SLTI/SLTUI); (3) 移位立即数 (SLLI/SRLI/SRAI); (4) LU12I.W / PCADDU12I; (5) 分支比较 (BEQ/BNE/BLT/BGE/BLTU/BGEU) → branch_taken; (6) 无条件跳转 (B/BL/JIRL) → jump_taken + jump_addr; (7) ERTN 异常返回; (8) CSR 读回值写入 GPR。
EX_LS	EX_LS.v	槽1 执行单元。功能: (1) ALU 运算 (同 EX_ALU 但不含分支/跳转); (2) Load/Store 地址生成 (rj + imm); (3) 访存宽度解析 (opcode[1:0]: 00=byte, 01=half, 10=word); (4) Store 数据直通 (reg2 → mem_wdata); (5) CSR 读回值写入 GPR。

前递网络 (Forwarding):

EX 级通过组合逻辑前递解决 RAW 数据冒险。优先级: ls_mem > br_mem > ls_wb > br_wb。

- **GPR 前递:** EX 级 4 个源操作数 (槽0 rj/rk、槽1 rj/rk) 与 MEM 级和 WB 级的目标寄存器地址比较, 命中则旁路最新数据。
- **CSR 前递:** 流水线中未完成的 CSR 写如地址匹配, 则前递 CSR 写入数据。优先级: br_wb > br_mem > br_ex > CSR 组合读。
- **ERA/PRMD 前递:** 支持 ERTN 指令所需的前递。

ALE 异常检测:

访存指令 (memRead/memWrite) 且地址未对齐时在 EX→MEM 流水线寄存器中标记 ALE 异常 (ecode=0x09)。

3.4 MEM 阶段 (访存)

模块	文件	主要功能
MEM_Stage	MEM_Stage.v	访存阶段封装，内嵌 dCache。输入：访存有效信号、写使能、访存宽度、地址（物理地址 <code>mem_pa</code> ）、写数据（对齐移位后）、字节写使能。输出：读数据（符号/零扩展后）、 <code>cpu_stall</code> 。
dCache	dCache.v	2 路组相联数据缓存，Write-Back + Write-Allocate。64B 行大小，数据宽度 32b。Tag 含 valid + dirty 位。读命中 0 stall，写命中 1 stall（需 RMW），缺失触发逐出（脏行写回）+ 逐字填充。LRU 替换策略。
MMU (MEM 侧)	mmu.v	将 <code>memAddr_ls_mem</code> 虚地址翻译为物理地址 <code>mem_pa</code> 。

MEM 级 Store 数据前递：

Store 指令的源数据可能来自上一拍同对中槽0的 ALU 结果，本拍通过 `fwd_mem_br` 检测并前递。

写数据对齐：Store 数据根据 `memSize` 和地址低两位做移位对齐（st.b 左移 0/8/16/24 bit，st.h 左移 0/16 bit）。

异常阻塞：当前 MEM 槽1或槽0（且槽1高位伴生）有异常时，`block_ls` 阻止访存操作进入 dCache。

读数据子字提取（WB 级组合逻辑）：

memSize	提取字节	符号扩展规则
00 (byte)	addr[1:0] 选 1 字节	signExt=0→符号扩展, 1→零扩展
01 (half)	addr[1] 选 2 字节	signExt=0→符号扩展, 1→零扩展
10 (word)	整 4 字节	无需扩展

3.5 WB 阶段（写回）

模块	文件	主要功能
Regs (写口)	Regs.v	WB 级写入通用寄存器。两槽通过 <code>regWrite_br_wb</code> / <code>regWrite_ls_wb</code> 控制写入，写入由 <code>flush</code> 和 <code>stall_delayed</code> 门控保护。槽1 写数据经 MUX 选择：load→dCache 读数据，非 load→ALU 结果。同周期双写冲突时 slot1 优先（后写覆盖）。
CSR (写口)	csr.v	WB 级写入 CSR（通过 <code>csrBus_br_wb</code> / <code>csrBus_ls_wb</code> 控制）。支持 ERTN（写 CRMD.PLV）、CSR 写/交换、异常处理（写 ERA/BADV/ESTAT/PRMD/CRMD）。

WB 级异常仲裁：

当两槽同时有异常时，取较老的指令（high=1 为高位伴生指令）：

- LS 的 high=1 → BR 更老 → 取 BR 异常
- BR 的 high=1 → LS 更老 → 取 LS 异常

写使能门控：`regWrite && !flush && !stall_delayed`，防止异常/冲刷/暂停时误写入。

4. 流水线以外模块设计

4.1 pipeline_controll（流水线控制）

文件：`pipeline_controll.v`

功能：统一管理流水线冲刷和暂停信号。

输出	触发条件	功能
stall_out	dCache stall 或 iCache stall	全局暂停，冻结所有流水线寄存器
flush_id	branch_taken 或 WB 级异常	冲刷 ID 级（指令变 NOP）
flush_ex	branch_taken 或 WB 级异常	冲刷 EX 级流水线寄存器
flush_br_mem	WB 级异常	冲刷槽0 MEM 级中继寄存器
flush_ls_mem	(branch_taken 且 LS 高位伴生) 或 WB 级异常	冲刷槽1 MEM 级寄存器
flush_br_wb	异常冲刷（考虑 high 位仲裁）	冲刷槽0 WB 写回
flush_ls_wb	异常冲刷（考虑 high 位仲裁）	冲刷槽1 WB 写回

4.2 axi_bridge（AXI 转接桥）

文件：axi_bridge.v

功能：将 iCache 和 dCache 的外部请求转接到 AXI4 总线。

关键设计：

- 单拍传输（arlen/awlen=0），无 burst
- 仲裁：dCache 优先于 iCache（d_take = d_ext_req, i_take = i_ext_req && !d_ext_req）
- AXI ID：取指=0，存取=1
- 状态机：6 状态
 - S_IDLE → 仲裁、确定事务类型
 - S_AR_REQ → 发起读地址请求 → S_R_DATA → 接收读数据 → S_IDLE
 - S_AW_REQ → 发起写地址请求 → S_W_DATA → 发送写数据 → S_B_RESP → 等待写响应 → S_IDLE
- Cache 侧接口：ext_ready 握手协议，单拍直通，无缓冲

4.3 CSR（控制状态寄存器）

文件：csr.v

功能：实现 LoongArch 特权架构定义的 CSR 寄存器组。

实现的 CSR：

CSR	编号	描述
CRMD	0x0	当前模式信息：PLV[1:0]、IE、DA、PG、DATF、DATM
PRMD	0x1	例外前模式：PPLV、PIE
ECFG	0x4	例外配置：局部中断使能
ESTAT	0x5	例外状态：IS[12:0]、Ecode、EsubCode
ERA	0x6	例外返回地址
BADV	0x7	出错虚地址
EENTRY	0xC	异常入口地址
LLBCTL	0x60	LLBit 控制
TID	0x40	定时器编号
TCFG	0x41	定时器配置
TVAL	0x42	定时器当前值（28b 宽，TIMER_WIDTH=28）
TICLR	0x44	定时器清除（写清定时器，读恒为 0）

CSR	编号	描述
DMW0	0x180	直接映射窗口 0
DMW1	0x181	直接映射窗口 1
SAVE0-3	0x30-33	保存寄存器 (4 个)

关键功能：

- **双端口组合读出** (`raddr0 / raddr1`)，支持两槽同时读取不同 CSR
- **异常处理**： `except` 信号优先于 `wea`，自动写 ERA、BADV、ESTAT 的 Ecode 字段、CRMD (PLV←0, IE←0)、PRMD (保存旧 PLV/IE)
- **中断检测**： `int_pending = CRMD.IE & (!(ESTAT.IS & ECFG.LIE))`，支持 HWI[7:0]、IPI、TI
- **ERTN 支持**：输出 `era_val`、`prmd_val`、`eentry_val` 供流水线使用

4.4 MMU + TLB (地址翻译)

文件： `mmu.v` + `tlb.v`

MMU 功能：

- **DA=1 (直接地址翻译)**： `pa = va` (直通)
- **PG=1 (映射地址翻译)**：先查 DMW0/DMW1，命中则 `pa = {DMW.PSEG, va[28:0]}`；不命中暂直通 (待 TLB 完善)
- 两个翻译端口：IF 取指 (`if_va→if_pa`) 和 MEM 访存 (`mem_va→mem_pa`)

DMW 窗口命中条件： `VA[31:29] == DMW.VSEG 且 (plv==0 && DMW[0]) || (plv==3 && DMW[3])`

TLB 功能 (全相联, 16 项)：

- 每项 64b： `{V,D,G, ASID[9:0], VPN[19:0], PPN[19:0], PLV[1:0], MAT[1:0], reserved[6:0]}`
- 组合逻辑查找，0 拍命中
- CSR 维护接口：TLBSRCH (查找)、TLBRD (读取)、TLBWR (随机写)、TLBFILL (重填)、INVTLB (全无效)
- 随机替换计数器 (简易 LFSR)

4.5 Regs (寄存器文件)

文件： `Regs.v`

功能：32×32b 通用寄存器文件。

- **读口**：4 端口组合逻辑读出 (2 槽 × 2 源操作数)，r0 硬连线为 0
- **写口**：2 端口下降沿写入 (满足时序要求)，写使能由 `regWrite` 和 `en` 门控。r0 不可写入。双写冲突时 slot1 优先 (后写覆盖)

4.6 inst_controll (指令控制)

文件： `inst_controll.v`

功能：PC 控制和指令对齐逻辑。

PC 控制优先级 (组合逻辑)： `except_taken > jump_taken > 正常递增`

指令拼接：

- 正常 8B 对齐取指： `dual_inst = dual_inst_raw`
- 跳转目标 4B 对齐： `dual_inst = {NOP, dual_inst_raw[63:32]}` (高半→低半，补 NOP)
- 上一拍有剩余指令： `dual_inst = {dual_inst_raw[31:0], left}`
- 跳转/异常后：重置 `take`、`left`

4.7 iCache (指令缓存)

文件：iCache.v

参数：2 路组相联，128 组（256 行），64B 行大小，64b 数据宽度

状态机：S_IDLE → S_DATA(命中) / S_FILL_REQ(缺失) → S_FILL_WR → S_RETRY → S_DATA/S_FILL_REQ

填充策略：每 2 次 32b AXI 读拼装 1 个 64b BRAM 字（half_buf 暂存低 32b），line fill 共需 8 个 BRAM 字（即 16 次 AXI 读）。

特点：只读（无 dirty 位、无逐出写回），Tag 仅含 valid+tag（TAG_ENTRY_WIDTH = 1+TAG_WIDTH）。

4.8 dCache（数据缓存）

文件：dCache.v

参数：2 路组相联，128 组（256 行），64B 行大小，32b 数据宽度

写策略：Write-Back + Write-Allocate

状态机：8 状态

状态	功能
S_IDLE	空闲，检测 hit/miss
S_DATA	读命中，交付数据
S_HIT_WR	写命中，RMW 提交
S_EVICT_RD	逐出脏行：逐字读取
S_EVICT_WR	逐出脏行：逐字写回 AXI
S_EVICT_DONE	逐出完成
S_FILL_REQ	填充：发起 AXI 读
S_FILL_WR	填充：写入 BRAM
S_RETRY	重试原始请求

Tag 格式：{valid, dirty, tag}（TAG_ENTRY_WIDTH = 2+TAG_WIDTH）

延迟总结：

- 读命中：0 stall
- 写命中：1 stall（RMW）
- 读缺失：fill 全程 stall (~35–70 拍)
- 写缺失：逐出 + fill + RMW

5. 异常与中断处理

异常优先级（ID 级）

1. **中断** (ecode=0x00)：优先级最高，中断待处理时两槽同时标记异常
2. **ADEF** (ecode=0x08)：PC 未 4 字节对齐
3. **SYSCALL** (ecode=0x0B)
4. **BREAK** (ecode=0x0C)
5. **INE** (ecode=0x0D)：非法指令
6. **ALE** (ecode=0x09)：访存地址未对齐（EX→MEM 级检测）

异常处理流程

1. WB 级检测异常 → except_taken=1

2. `inst_controll` 将 PC 重定向至 `eentry_val` (来自 CSR.EENTRY)
3. `pipeline_controll` 冲刷各流水级
4. CSR 自动保存: $ERA \leftarrow \text{异常指令 PC}$ 、 $BADV \leftarrow \text{出错虚地址}$ 、 $ESTAT.Ecode \leftarrow \text{异常号}$ 、 $PRMD \leftarrow \text{保存旧 PLV/IE}$ 、 $CRMD(PLV \leftarrow 0, IE \leftarrow 0)$
5. ERTN 指令恢复 PLV/IE 并跳转至 ERA

中断检测

- 每拍采样 HWI[7:0]、IPI、定时器信号
- `int_pending = CRMD.IE & (|(ESTAT.IS[12:0] & ECFG.LIE[12:0]))`
- 中断在 ID 级插入，优先级高于一切异常

6. 源文件清单

文件	层级	功能
<code>core_top.v</code>	顶层	CPU 封装, AXI 接口 + 调试端口
<code>top.v</code>	核心	CPU 主模块, 连接所有子模块和流水线寄存器
<code>IF_Stage.v</code>	IF	取指阶段, 例化 iCache
<code>iCache.v</code>	IF	2 路组相联指令缓存
<code>inst_controll.v</code>	IF	PC 控制 + 指令拼接
<code>Decoder.v</code>	ID	LA32 指令译码器
<code>ID_stage.v</code>	ID	发射控制、冲突检测、槽位分配
<code>EX_ALU.v</code>	EX	槽0 执行单元 (ALU/Branch/Jump/CSR)
<code>EX_LS.v</code>	EX	槽1 执行单元 (ALU/Load/Store)
<code>MEM_Stage.v</code>	MEM	访存阶段, 例化 dCache
<code>dCache.v</code>	MEM	2 路组相联数据缓存 (Write-Back)
<code>Regs.v</code>	ID/WB	32×32b 寄存器文件 (4R2W)
<code>csr.v</code>	WB	控制状态寄存器组
<code>mmu.v</code>	IF/MEM	地址翻译 (DA/DMW)
<code>tlb.v</code>	IF/MEM	全相联 TLB (16 项)
<code>pipeline_controll.v</code>	全局	流水线冲刷/暂停控制
<code>axi_bridge.v</code>	全局	AXI 转接桥 (iCache+dCache→AXI)